INVENTORY REORDER ALERT SYSTEM - ASSESSMENT REPORT

Assessment Submission Form

Project Name:	Inventory Reorder Alert System
Repository URL:	https://github.com/santunandi95/inventory-reorder-alert
Author:	santunandi95
Date:	2026-07-31

1. Approach Summary

Reads 'stock_data.csv' into validated dictionaries, safely handling missing, negative, or malformed data by logging warnings and applying fallback values. Compares current stock against thresholds to classify priorities into 'Critical' (<25% of threshold) vs 'Low' tiers, and calculates the required reorder quantity to reach a healthy level. Generates a clean category-grouped console report, prints a simulated email alert, and exports the results to a structured 'restock_report.csv' file.

2. Project Files Created

- inventory_reorder_alert.py: Main script containing core file handling, validation, logic, and email simulation.
- stock_data.csv: Warehouse stock CSV loaded with 25 test items (including standard values and edge-case samples).
- restock_report.csv: Machine-readable output generated containing the low-stock alert list.
- README.md: Detailed documentation of parameters, logic, approach, and future extension ideas.

3. Python Source Code (inventory_reorder_alert.py)

```
import csv
import os
import sys
import io
from datetime import datetime

if sys.stdout.encoding and sys.stdout.encoding.lower() not in ("utf-8", "utf-8-sig"):
    sys.stdout = io.TextIOWrapper(sys.stdout.buffer, encoding="utf-8", errors="replace")


INPUT_FILE        = "stock_data.csv"
OUTPUT_REPORT_CSV = "restock_report.csv"
CRITICAL_THRESHOLD_PCT = 0.25

def load_stock_data(filepath: str) -> tuple[list[dict], list[str]]:
    """
    Reads the CSV file and returns:
      - A list of cleaned stock-item dictionaries.
      - A list of warning strings for skipped / malformed rows.
```

```
    """
    items    : list[dict] = []
    warnings : list[str]  = []

    if not os.path.exists(filepath):
        print(f"[ERROR] File not found: {filepath}")
        sys.exit(1)

    with open(filepath, newline="", encoding="utf-8") as fh:
        reader = csv.DictReader(fh)


        required_cols = {"item_name", "current_quantity", "reorder_threshold"}
        if not required_cols.issubset(set(reader.fieldnames or [])):
            missing = required_cols - set(reader.fieldnames or [])
            print(f"[ERROR] Missing columns in CSV: {missing}")
            sys.exit(1)

        for row_num, row in enumerate(reader, start=2):
            item = parse_row(row, row_num, warnings)
            if item:
                items.append(item)

    return items, warnings


def parse_row(row: dict, row_num: int, warnings: list[str]) -> dict | None:
    """
    Validates and normalises a single CSV row.
    Returns a clean dict or None if the row must be skipped.
    """
    name = row.get("item_name", "").strip()
    if not name:
        warnings.append(f"Row {row_num}: Skipped ? missing item name.")
        return None

    # Parse current_quantity
    try:
        qty = float(row.get("current_quantity", "").strip())
        if qty < 0:
            warnings.append(f"Row {row_num} ({name}): Negative quantity treated as 0.")
            qty = 0.0
    except (ValueError, AttributeError):
        warnings.append(f"Row {row_num} ({name}): Missing/invalid quantity ? treated as 0.")
        qty = 0.0

    # Parse reorder_threshold
    try:
        threshold = float(row.get("reorder_threshold", "").strip())
        if threshold <= 0:
            warnings.append(f"Row {row_num} ({name}): Non-positive threshold ? row skipped.")
            return None
    except (ValueError, AttributeError):
        warnings.append(f"Row {row_num} ({name}): Missing/invalid threshold ? row skipped.")
        return None
```

```
    # Parse healthy_stock_level (optional)
    try:
        healthy = float(row.get("healthy_stock_level", "").strip())
    except (ValueError, AttributeError):
        healthy = threshold * 3

    return {
        "item_name"          : name,
        "current_quantity"   : qty,
        "reorder_threshold"  : threshold,
        "healthy_stock_level": healthy,
        "unit"               : row.get("unit", "units").strip() or "units",
        "category"           : row.get("category", "Uncategorised").strip() or "Uncategorised",
    }



def classify_item(item: dict) -> dict | None:

    qty       = item["current_quantity"]
    threshold = item["reorder_threshold"]
    healthy   = item["healthy_stock_level"]

    if qty >= threshold:
        return None   # stock is fine

    critical_mark = threshold * CRITICAL_THRESHOLD_PCT
    priority      = "? CRITICAL" if qty < critical_mark else "? LOW"

    # BONUS: Reorder quantity suggestion
    reorder_qty = max(0, healthy - qty)

    return {
        **item,
        "priority"   : priority,
        "reorder_qty": int(reorder_qty),
        "shortage"   : int(threshold - qty),
    }


def analyse_stock(items: list[dict]) -> list[dict]:
    """Runs classify_item over every item; returns only those needing restock."""
    return [result for item in items if (result := classify_item(item)) is not None]


DIVIDER  = "?" * 72
THIN_DIV = "?" * 72

def print_console_report(flagged: list[dict], warnings: list[str], run_time: str) -> None:
    """Prints a formatted restock report to the console."""

    critical_items = [i for i in flagged if "CRITICAL" in i["priority"]]
    low_items      = [i for i in flagged if "LOW"      in i["priority"]]

    print(f"\n{DIVIDER}")
    print(f"  [BOX] INVENTORY REORDER ALERT REPORT")
```

```
    print(f"  Generated : {run_time}")
    print(f"  Input     : {INPUT_FILE}")
    print(DIVIDER)
    print(f"  Items needing restock : {len(flagged)}")
    print(f"  [!!] Critical         : {len(critical_items)}")
    print(f"  [!]  Low              : {len(low_items)}")
    print(DIVIDER)

    if not flagged:
        print("\n  [OK] All stock levels are above their reorder thresholds. No action needed.\n")
        return

    # Group by category for readability
    categories = {}
    for item in flagged:
        categories.setdefault(item["category"], []).append(item)

    for category, cat_items in sorted(categories.items()):
        print(f"\n  [FOLDER] {category.upper()}")
        print(f"  {THIN_DIV}")
                print(f"    {'Item':<30}  {'Priority':<12}  {'Qty':>6}  {'Threshold':>10}  {'Reorder':>8}
{'Unit':<10}")
        print(f"  {THIN_DIV}")
        for i in sorted(cat_items, key=lambda x: x["current_quantity"]):
            priority_label = "[!!] CRITICAL" if "CRITICAL" in i["priority"] else "[!]  LOW"
            print(
                f"  {i['item_name']:<30} {priority_label:<12} "
                f"{int(i['current_quantity']):>6} {int(i['reorder_threshold']):>10} "
                f"{i['reorder_qty']:>8} {i['unit']:<10}"
            )

    if warnings:
        print(f"\n  {THIN_DIV}")
        print(f"  [!] DATA WARNINGS ({len(warnings)})")
        print(f"  {THIN_DIV}")
        for w in warnings:
            print(f"  * {w}")

    print(f"\n{DIVIDER}\n")



def format_email_alert(flagged: list[dict], run_time: str) -> str:
    """
    Returns a string that mimics a restock-alert email
    (Subject + Body) ? ready to hand to smtplib or any mailer.
    """
    critical_items = [i for i in flagged if "CRITICAL" in i["priority"]]
    low_items      = [i for i in flagged if "LOW"      in i["priority"]]

    subject = (
        f"[RESTOCK ALERT] {len(flagged)} items need attention "
        f"({len(critical_items)} CRITICAL) ? {run_time[:10]}"
    )

    lines = [
```

```
        "=" * 60,
        f"  TO      : warehouse-manager@company.com",
        f"  FROM    : inventory-bot@company.com",
        f"  SUBJECT : {subject}",
        "=" * 60,
        "",
        "Hello,",
        "",
        f"This is your automated daily inventory check for {run_time[:10]}.",
        f"A total of {len(flagged)} item(s) are below their reorder threshold.",
        "",
    ]


    if critical_items:
        lines.append("[!!] CRITICAL -- Order Immediately:")
        lines.append("-" * 40)
        for i in critical_items:
            lines.append(
                f"  * {i['item_name']}: {int(i['current_quantity'])} {i['unit']} remaining "
                f"(threshold {int(i['reorder_threshold'])}). "
                f"Suggest ordering {i['reorder_qty']} {i['unit']}."
            )
        lines.append("")

    if low_items:
        lines.append("[!] LOW -- Plan to Reorder Soon:")
        lines.append("-" * 40)
        for i in low_items:
            lines.append(
                f"  * {i['item_name']}: {int(i['current_quantity'])} {i['unit']} remaining "
                f"(threshold {int(i['reorder_threshold'])}). "
                f"Suggest ordering {i['reorder_qty']} {i['unit']}."
            )
        lines.append("")

    lines += [
        "Please coordinate with the relevant suppliers as soon as possible.",
        "",
        "This is an automated message ? do not reply directly.",
        "Inventory Alert System v1.0",
        "=" * 60,
    ]

    return "\n".join(lines)



def export_csv_report(flagged: list[dict], filepath: str, run_time: str) -> None:
    """Writes the flagged items to a restock_report.csv file."""
    fieldnames = [
        "item_name", "category", "priority",
        "current_quantity", "reorder_threshold",
        "shortage", "reorder_qty", "healthy_stock_level",
        "unit", "report_generated",
    ]
```

```
    with open(filepath, "w", newline="", encoding="utf-8") as fh:
        writer = csv.DictWriter(fh, fieldnames=fieldnames, extrasaction="ignore")
        writer.writeheader()
        for item in flagged:
            row = {**item, "report_generated": run_time}
            # Strip emoji from priority for clean CSV
            row["priority"] = row["priority"].replace("? ", "").replace("? ", "")
            writer.writerow(row)

    print(f"  [OK] CSV report exported -> {filepath}")



def main() -> None:
    run_time = datetime.now().strftime("%Y-%m-%d %H:%M:%S")

    print(f"\n  [*] Running inventory check at {run_time} ...")

    # Step 1 -- Load data
    items, warnings = load_stock_data(INPUT_FILE)
    print(f"  [*] Loaded {len(items)} valid item(s) from '{INPUT_FILE}'.")

    # Step 2 -- Analyse
    flagged = analyse_stock(items)

    # Step 3 -- Console report
    print_console_report(flagged, warnings, run_time)

    # Step 4 -- Email simulation (always print so the output is visible)
    if flagged:
        email_body = format_email_alert(flagged, run_time)
        print("\n  [EMAIL] SIMULATED EMAIL ALERT")
        print(email_body)
    else:
        print("  [EMAIL] No email alert needed -- all stock levels are healthy.\n")

    # Step 5 -- Export CSV
    export_csv_report(flagged, OUTPUT_REPORT_CSV, run_time)


if __name__ == "__main__":
    main()
```

INVENTORY REORDER ALERT SYSTEM - ASSESSMENT REPORT

4. Sample Input Data (stock_data.csv)

```
item_name,current_quantity,reorder_threshold,unit,healthy_stock_level,category
Nitrile Gloves (Box),12,50,boxes,200,Safety
Safety Goggles,3,20,units,80,Safety
Hard Hats,45,30,units,120,Safety
Steel-Toe Boots,0,15,pairs,60,Safety
Packing Tape,8,40,rolls,160,Packaging
Bubble Wrap,2,25,rolls,100,Packaging
Cardboard Boxes (Large),150,100,units,400,Packaging
Cardboard Boxes (Small),500,200,units,800,Packaging
Shipping Labels,1000,500,sheets,2000,Packaging
Stretch Film,5,30,rolls,120,Packaging
Forklift Batteries,1,4,units,16,Equipment
Pallet Jacks,6,3,units,12,Equipment
Hand Trucks,10,5,units,20,Equipment
Barcode Scanners,2,5,units,20,Equipment
Printer Ink Cartridges,3,10,cartridges,40,Office
A4 Paper (Ream),25,50,reams,200,Office
Pens (Box),0,5,boxes,20,Office
Cleaning Spray,7,20,bottles,80,Maintenance
Mop Heads,4,10,units,40,Maintenance
Trash Bags (Roll),11,30,rolls,120,Maintenance
First Aid Kits,2,8,kits,32,Safety
Fire Extinguishers,10,10,units,40,Safety
,50,20,units,80,Miscellaneous
WD-40 Lubricant,,15,cans,60,Maintenance
Zip Ties (Pack),200,100,packs,400,Packaging
```

5. Output Generated Report (restock_report.csv)

```
item_name,category,priority,current_quantity,reorder_threshold,shortage,reorder_qty,healthy_stock_level,unit,report_generated
Nitrile Gloves (Box),Safety,CRITICAL,12.0,50.0,38,188,200.0,boxes,2026-07-31 14:40:28
Safety Goggles,Safety,CRITICAL,3.0,20.0,17,77,80.0,units,2026-07-31 14:40:28
Steel-Toe Boots,Safety,CRITICAL,0.0,15.0,15,60,60.0,pairs,2026-07-31 14:40:28
Packing Tape,Packaging,CRITICAL,8.0,40.0,32,152,160.0,rolls,2026-07-31 14:40:28
Bubble Wrap,Packaging,CRITICAL,2.0,25.0,23,98,100.0,rolls,2026-07-31 14:40:28
Stretch Film,Packaging,CRITICAL,5.0,30.0,25,115,120.0,rolls,2026-07-31 14:40:28
Forklift Batteries,Equipment,LOW,1.0,4.0,3,15,16.0,units,2026-07-31 14:40:28
Barcode Scanners,Equipment,LOW,2.0,5.0,3,18,20.0,units,2026-07-31 14:40:28
Printer Ink Cartridges,Office,LOW,3.0,10.0,7,37,40.0,cartridges,2026-07-31 14:40:28
A4 Paper (Ream),Office,LOW,25.0,50.0,25,175,200.0,reams,2026-07-31 14:40:28
Pens (Box),Office,CRITICAL,0.0,5.0,5,20,20.0,boxes,2026-07-31 14:40:28
Cleaning Spray,Maintenance,LOW,7.0,20.0,13,73,80.0,bottles,2026-07-31 14:40:28
Mop Heads,Maintenance,LOW,4.0,10.0,6,36,40.0,units,2026-07-31 14:40:28
Trash Bags (Roll),Maintenance,LOW,11.0,30.0,19,109,120.0,rolls,2026-07-31 14:40:28
First Aid Kits,Safety,LOW,2.0,8.0,6,30,32.0,kits,2026-07-31 14:40:28
```

```
WD-40 Lubricant,Maintenance,CRITICAL,0.0,15.0,15,60,60.0,cans,2026-07-31 14:40:28
```

6. Assessment Reflection & Future Scope

Given more time, the following improvements would be integrated:

1. Scheduling: Establish automated execution triggers via system Cron or Windows Task Scheduler.
2. Real Notification Service: Integrate SMTP relay or APIs like SendGrid/SES to dispatch warning digests.
3. Supplier API/Integrations: Directly send draft Purchase Orders to pre-mapped vendors for low items.
4. Historical Tracking & Smart Reorder Thresholds: Aggregate daily snapshots into a database to detect trends, calculate moving velocity, and adjust thresholds automatically to prevent stockouts.